Dev: AI, search and data work doc

Oct 24, 2026 · @Chaudhari kevin

Mission and setup

You make Rally understand people and the real world: Gemini compiles goals and explains plans, Moss ranks matches and searches vendors, and Apify brings in real listed prices. Every feature stays behind a flag, so `main` still runs offline when a key or the network is missing.

- Branch: `feat/ai-search-data`, from `main` after Kevin's contracts PR (target 3:30 pm).
- You own: `services/api/rally/compiler.py`, `services/api/rally/matching.py`, a new `services/api/rally/ai/` package, `data/`, `tests/evaluation/`. Add service methods in a marked block at the end of `service.py`.
- Don't edit: `apps/web/`, `convex/`, `negotiation/`, `agents/`. Ask Kevin for UI; contract changes go through Kevin.

Setup:

```
git checkout main && git pull && git checkout -b feat/ai-search-data
python scripts/rally.py setup
pip install google-genai moss apify-client
# .env (never committed)
GEMINI_API_KEY=...        # Google AI Studio
GEMINI_MODEL=gemini-2.5-flash   # check the current model id first
MOSS_PROJECT_ID=...  MOSS_PROJECT_KEY=...   # moss.dev, free tier
APIFY_TOKEN=...
python scripts/rally.py test
```

What exists already: `compiler.py` has the full pipeline (sanitize, extract, resolve dates, assemble with `missing` questions) and an untested `extract_gemini()` that sends the flat `Extracted` schema as Gemini's response schema. `matching.py` has the hard filters and a `LocalRanker` with a `rank(seeker, pool) -> {intent_id: score}` interface. `service.translate()` already calls Gemini for `/v1/translate` when a key is set. The activity vocabulary is in `data/fixtures/activities.json`.

Tasks, in this order

#	Task	Priority	Hours	Due
1	Gemini goal compiler live + 30-goal eval	P0	4	9 pm, merge 1
2	"Why this plan works for you"	P1	3	2 am, merge 2
3	Moss ranking + vendor search	P1	3	2 am
4	Apify listed prices	P1	3	2 am
5	Vendor reply suggestions	P2	2	only if 1 to 4 are merged

1. Gemini goal compiler

Goal: the review page shows "Compiled by Gemini" for any of the three activities, and Gemini never invents a budget, date or place.

1. Run `extract_gemini()` against the real API for the five example goals. Check the current `google-genai` structured-output syntax (`response_schema` with the Pydantic `Extracted` model, or a JSON schema) and the model id.
2. Keep the flat `Extracted` schema. Gemini fills only what the user stated, plus `template_id`, `activity` (must be a key in `activities.json`) and `date_phrase` copied verbatim. Deterministic code still resolves dates and builds `missing` questions.
3. Guard rails already in `compile_goal()`: one retry, then the template compiler, then `compiledBy: "template"` and a warning. Add a check that a stated budget's number appears in the text.
4. Write `tests/evaluation/compiler_goals.jsonl` with 30 goals: 10 scuba, 6 skydiving, 6 kayaking, 4 other templates (tennis, ski, carpool, group buy), 2 vague ("diving soon"), 2 injection attempts. Each line: the text, a fixed today (2026-10-24), expected `template_id`, `activity`, hard conditions and missing fields.
5. Write `tests/evaluation/run_compiler.py` that scores both compilers and saves `tests/evaluation/results/compiler.json`: template accuracy, hard-condition exact match, invented-field rate (target 0).

Test: unit tests stay offline (mock Gemini's response); the eval is the live check.

Done when `COMPILER=gemini` compiles all three demo goals and the eval numbers are committed.

2. "Why this plan works for you"

Goal: each member sees two sentences on how the final plan fits them, without their private preferences leaving their own agent.

1. Parikshit adds `PersonalAgentBrain.explain(plan)`, which returns per-term fit only: `[{term, value, fit: "great" | "ok" | "compromise"}]`, no weights or scores.
2. New `services/api/rally/ai/explain.py`: Gemini turns that list plus the public plan into two sentences, for example "You got your Point Lobos boat dive. The date moved to Sunday, which you said was fine." Template fallback without a key.
3. `GET /v1/deals/{id}/explanation` returns only the caller's own text. Never put it in the shared snapshot.

Test: the endpoint returns 403 for non-members, and no weight values appear in the response.

3. Moss ranking and vendor search

1. New `services/api/rally/ai/moss_ranker.py` with the same `rank(seeker, pool)` interface as `LocalRanker`. On publish, upsert each intent into an index `rally-intents` (text = activity, tags, place, month); on match, query with the seeker's text, `top_k=50`, and keep only Stage 1 survivors. Similarity never admits a rejected person.
2. `RANKER=moss` picks it; on any error, fall back to `LocalRanker` and set `ranker: "local"` so the label stays honest.
3. Second index `rally-vendors` from `providers.json` (name, activity, place, offers, includes). New `GET /v1/providers/search?q=` returns ranked vendors with scores, for the vendor search box Kevin adds on the home page.

Test: with a fake Moss client, a hard-rejected person never appears in matches.

4. Apify listed prices

1. Pick 3 to 6 public pricing pages for real Monterey-area dive, skydive or kayak shops. Run an Apify Actor (a website content crawler) ahead of time, not during the demo.
2. `data/scripts/import_listed_prices.py` parses each price into the existing tier shape and saves `data/fixtures/listed_prices.json` with `provenance: "cached"`, the source URL and `lastVerifiedAt`.
3. Load these as extra providers with `source: "listed"`. They show as "Listed price, not a quote", are never the binding quote, and have no agent.

Check each site's terms before scraping, and use only public price pages. If a parse is ambiguous, leave that shop out rather than guess.

5. Vendor reply suggestions (P2)

`POST /v1/vendor/requests/{id}/suggest` returns a Gemini-drafted reply to the latest group message in the vendor's chosen language. Kevin adds a "Suggest reply" button above the vendor chat box; staff edit it before sending.

Handoffs, fallbacks and checklist

What	To	When	How
`/v1/deals/{id}/explanation` response shape	Kevin (card in the group room)	11 pm	`{text, generatedBy: "gemini" or "template"}`
`/v1/providers/search?q=` response shape	Kevin (search box)	Midnight	`{results: [{providerId, name, activity, score}], ranker}`
Listed-price providers	Kevin (provider page badge)	2 am	Already labelled by `provenance: "cached"`
`PersonalAgentBrain.explain(plan)`	From Parikshit	9 pm	Per-term fit list, no weights
Compiler and eval numbers for the pitch	Kevin	2 am	`tests/evaluation/results/compiler.json`

Fallbacks:

- No Gemini key or quota: the template compiler covers the three demo goals; the review page says "Compiled offline".
- Moss down: `LocalRanker`, labelled "Ranked locally".
- Apify Actor fails or a site blocks it: skip listed prices; seeded vendors carry the demo.

Final checklist:

- [ ] `python scripts/rally.py test` passes on `feat/ai-search-data`, with Gemini, Moss and Apify mocked
- [ ] `COMPILER=gemini` works on the scuba, skydiving and kayaking goals
- [ ] 30-goal eval committed; invented-field rate is 0
- [ ] `RANKER=moss` shows "Ranked by Moss" and never admits a hard-rejected person
- [ ] Explanation endpoint returns only the caller's own text
- [ ] Listed prices labelled "Listed price, not a quote", each with a source link
- [ ] Every new key is in `.env.example` (names only)

Moss SDK facts are from its project page: Python `pip install moss`, `create_index`, `add_docs(..., upsert=True)`, `query(name, text, top_k)`, project id and key from moss.dev.